

Lab 14 · Cuatro oficinas en vez de una

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

180 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · tres PostgreSQL embebidos

Resumen

Que partir un sistema en servicios resuelve unos problemas y estrena otros. Con un servicio caído, la misma petición pasa de **500 a 200 degradado** — y el mismo identificador aparece en las tres consolas.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
1.4. La puesta a punto	3
2. El caso	3
2.1. Cuatro oficinas, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · Levantar el sistema	4
3.2. Paso 2 · Una base por servicio	5
3.3. Paso 3 · La llamada entre servicios	5
3.4. Paso 4 · Matar un servicio — el momento fuerte	7
3.5. Paso 5 · Circuit breaker y degradación	8
3.6. Paso 6 · El gateway	9
3.7. Paso 7 · Seguir una petición por tres servicios	11
3.8. Paso 8 · Consistencia eventual	12
4. Lo que aprendiste	13
5. Para profundizar	13
6. Antes de cerrar	14

1 Antes de empezar

1.1 Qué vas a lograr

Todo lo que has hecho en trece laboratorios era **una aplicación**. Hoy la DGT se parte en **cuatro procesos con tres bases de datos**, y aparecen los problemas que sólo existen cuando el trabajo está repartido.

Vas a levantar los cuatro, vas a ver por qué cada uno tiene **su propia base**, vas a hacer que uno llame a otro, **vas a matar un servicio a propósito** y verás que el sistema sigue en pie, vas a poner una puerta única delante, y vas a seguir una petición por tres consolas con el mismo identificador.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 09, 10 y 11 hechos	Aquí se usan token, circuito y trace id, todos juntos	Muy recomendable
Estar en la carpeta del lab	cd labs/lab-14-microservicios/practica	El cd no da error
Cuatro terminales	Una por servicio	Es el lab que más ventanas abre
Memoria libre	Tres PostgreSQL a la vez	Cierra lo que no uses

Este laboratorio es distinto de los trece anteriores. No hay un `practica/` con un `mvnw`: hay **cuatro carpetas**, una por servicio, y cada una se arranca por separado en su propia terminal.

Puertos: **gateway 8200**, contribuyentes 8201, trámites 8202, auditoría 8203. Y tres PostgreSQL, uno por servicio.

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en `labs/lab-14-microservicios/solucion/`.

1.4 La puesta a punto

Cuatro terminales, y en cada una:

```
cd labs/lab-14-microservicios/practica/contribuyentes && ./mvnw spring-boot:run
cd labs/lab-14-microservicios/practica/tramites && ./mvnw spring-boot:run
cd labs/lab-14-microservicios/practica/auditoria && ./mvnw spring-boot:run
cd labs/lab-14-microservicios/practica/gateway && ./mvnw spring-boot:run
```

Arranca contribuyentes primero y el gateway el último. No es obligatorio —cada uno arranca solo—, pero así las primeras llamadas no fallan mientras los demás terminan de subir.

2 El caso

La DGT ha crecido. Una sola oficina con una sola cola atiende contribuyentes, tramita expedientes y lleva el registro de auditoría. **El día que hay que ampliar la mesa de trámites, hay que cerrar la oficina entera.**

2.1 Cuatro oficinas, que es la metáfora de este laboratorio

La DGT se parte en oficinas especializadas, cada una con su propio archivador.

- La **Oficina de Contribuyentes** tiene el padrón. Su archivador es suyo.
- La **Oficina de Trámites** tramita. Tiene su propio archivador de expedientes, y **no puede abrir el de contribuyentes**: si necesita un nombre, **llama por teléfono** y lo pide.
- La **Oficina de Auditoría** lleva el registro de lo que pasa.
- Y en la calle hay **una recepción**: el ciudadano no va a cada oficina, va a la recepción, enseña su carnet **una sola vez**, y le dirigen.

Que cada oficina tenga **su propio archivador** es lo que hace que esto sea de verdad cuatro oficinas y no un mostrador con cuatro ventanillas. Se puede reformar una sin cerrar las otras. Y se paga: **ya no hay un JOIN**. Cuando Trámites necesita el nombre de un contribuyente, tiene que llamar. Y llamar puede fallar — lo que en una sola aplicación era imposible, aquí es la norma.

Lo que hay que decidir es qué pasa cuando la otra oficina no contesta. Ésa es la pregunta del laboratorio.

3 Los pasos

3.1 Paso 1 · Levantar el sistema

Qué vamos a hacer

Arrancar los cuatro y comprobar que están en pie.

Para entenderlo mejor

Abrir las cuatro oficinas. Cada una con su llave, su archivador y su horario.

El problema

Cuatro procesos son cuatro cosas que arrancar, cuatro que vigilar y cuatro que pueden fallar por separado. **Eso es el coste de entrada**, y conviene sentirlo antes de hablar de las ventajas.

Se corre

Cada servicio en su terminal, y después:

```
curl localhost:8200/salud
curl localhost:8201/salud
curl localhost:8202/salud
curl localhost:8203/salud
```

Lo que vas a ver

```
{"servicio":"gateway","estado":"vivo"}
{"servicio":"contribuyentes","estado":"vivo"}
{"servicio":"tramites","estado":"vivo"}
{"servicio":"auditoria","estado":"vivo"}
```

Vas bien si...

Los cuatro contestan "estado":"vivo", cada uno diciendo su nombre.

Si te atascas

1 · EL PUERTO 554XX YA ESTA OCUPADO

Los mismos candados del Lab 04, ahora por triplicado: cada servicio tiene su PostgreSQL. El mensaje dice qué puerto:

```
lsof -ti:55450 | xargs kill -9
```

2 · ESTE MISMO PROYECTO YA ESTA CORRIENDO

Lo tienes arrancado en otra terminal. Con cuatro servicios es fácil perder la cuenta: revisa las cuatro antes de matar nada.

3 · Uno de los cuatro no arranca y los otros sí.

Es lo normal en un sistema repartido, y conviene notarlo: **el sistema no arranca o no arranca de golpe**. Mira la terminal de ése.

4 · La máquina se queda sin memoria o va muy lenta.

Tres PostgreSQL más cuatro JVM. Cierra el navegador y lo que no estés usando.

3.2 Paso 2 · Una base por servicio

Qué vamos a hacer

Mirar que cada servicio tiene su propia base, y entender qué se gana y qué se pierde.

Para entenderlo mejor

Cada oficina, su archivador. Trámites **no tiene llave** del de Contribuyentes.

El problema

Si dos servicios comparten tablas, un ALTER TABLE de uno rompe al otro. Y entonces **no se pueden desplegar por separado** — que era todo el motivo de partirlos. Lo que tendrías serían dos procesos con un acoplamiento invisible, que es lo peor de los dos mundos.

La alternativa, y por qué no

- **Una sola base compartida:** cómoda, permite JOIN, y anula la independencia. Si es lo que quieres, conviene admitir que tienes un monolito y ahorrarte los cuatro procesos.
- **Un esquema por servicio dentro de una base:** el término medio razonable cuando operar tres bases es demasiado. Mantiene la separación lógica y comparte la operación.
- **Una base por servicio,** que es lo de aquí: independencia de verdad, y se paga con lo que viene a continuación.

Lo que se pierde, y hay que decirlo entero:

- **No hay JOIN entre servicios.** El nombre del contribuyente hay que **pedirlo**.
- **No hay transacción que abarque a los dos.** Si Trámites guarda y Auditoría falla, no hay rollback que deshaga las dos cosas.
- **La consistencia deja de ser instantánea.** Durante un rato, dos servicios pueden tener versiones distintas de la verdad.

Vas bien si...

Puedes decir en qué puerto está la base de cada servicio, y por qué Trámites no puede consultar la tabla de contribuyentes directamente.

3.3 Paso 3 · La llamada entre servicios

Qué vamos a hacer

Que Trámites pida a Contribuyentes el nombre del titular.

Para entenderlo mejor

La llamada telefónica entre oficinas. Lo que antes era mirar en el cajón de al lado, ahora es marcar un número.

El problema

El expediente guarda el **RUT**, no el nombre. Y el nombre está en otra oficina.

La alternativa, y por qué no

- **Duplicar el nombre en la tabla de trámites**: rapidísimo de leer, y **se queda viejo** en cuanto alguien cambie de razón social. Se hace, y hay que asumir el desfase.
- **Feign** o `@HttpExchange`: se declara una interfaz y la implementación se genera. Menos ruido, y esconde la llamada — que en este lab es justo lo que hay que ver.
- **RestClient a pelo**, que es lo de aquí: se ve la URL, se ve el timeout, se ve el catch. Y lo que este lab enseña es **qué pasa cuando el otro lado no responde**, así que hay que poder tocarlo.

Se pega

La tabla de rutas la verás en el paso 6. Aquí, el cliente que llama —en `practica/tramites/src/main/java/cl/dgt`

```
/**
 * La ficha del contribuyente, o vacío si el servicio no pudo contestar.
 * Devolver `Optional` y no lanzar es la decisión: quien llama tiene que elegir qué
 *   ↪ hacer.
 */
public Optional<FichaDto> ficha(String rut) {
    try {
        return Optional.ofNullable(circuito.executeCallable(() -> pedir(rut)));
    } catch (CallNotPermittedException e) {
        log.warn("[TRAMITES] circuito ABIERTO: no llamo a contribuyentes por {}", rut);
        return Optional.empty();
    } catch (Exception e) {
        log.warn("[TRAMITES] contribuyentes no contestó por {}: {}", rut,
        ↪ e.getClass().getSimpleName());
        return Optional.empty();
    }
}
```

Fíjate en que devuelve Optional y no lanza. Es la decisión del laboratorio: el cliente no decide qué hacer cuando el otro falla — **eso lo decide quien llama**, que es el que conoce el trámite.

Se corre

```
TOKEN=$(curl -s -X POST http://localhost:8200/auth/login \
-H 'Content-Type: application/json' \
-d '{"usuario":"carolina","clave":"dgt2026"}' | sed
↪ 's/.*"token": "\([^"]*\)".*/\1/')

curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8200/tramites/1
```

Lo que vas a ver

```
{ "id":1, "tipo":"DECLARACION_F29", "estado":"EN_PROCESO",  
  "rutContribuyente":"11111111-1",  
  "nombreContribuyente":"Carolina Fuentes Aravena",  
  "estadoDelNombre":"OK",  
  "creadoEn":"2026-08-29T05:23:52.619319Z" }
```

El nombreContribuyente no está en la base de Trámites. Vino de una llamada HTTP al otro servicio, y ese estadoDelNombre: OK está ahí para el paso siguiente.

Vas bien si...

El trámite trae el nombre del contribuyente y "estadoDelNombre":"OK".

Si te atascas

1 · "nombreContribuyente":null y estadoDelNombre distinto de OK.

Contribuyentes no está arrancado, o la URL del application.yml apunta al puerto equivocado.

2 · 401 en todo.

El token no se recogió bien. Comprueba que la variable TOKEN no está vacía: echo \$TOKEN.

3.4 Paso 4 · Matar un servicio — el momento fuerte

Qué vamos a hacer

Cerrar Contribuyentes con Ctrl+C y volver a pedir el mismo trámite.

Para entenderlo mejor

La Oficina de Contribuyentes ha cerrado por una avería. ¿Qué hace la de Trámites? Cerrar también, o atender explicando lo que no puede confirmar.

El problema

En una sola aplicación esto no podía pasar: llamar a otra clase no falla. Aquí, **cada llamada entre oficinas es una llamada de red**, y la red falla. Es la diferencia esencial entre un sistema repartido y uno que no lo es.

Lo que vas a ver

Ve a la terminal de contribuyentes y para el servicio con Ctrl+C. Después:

```
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8200/tramites/1
```

Sin ninguna defensa, esto es lo que sale:

```
{"timestamp":"2026-08-29T05:28:08.644Z","status":500,"error":"Internal Server  
↪ Error","path":"/tramites/1"}  
<- HTTP 500
```

Un 500. El trámite existe, está en la base de Trámites, y no se puede consultar — porque **otra oficina** está cerrada. Eso es una dependencia mal resuelta: has convertido la caída de un servicio en la caída de dos.

Vas bien si...

Con contribuyentes parado, la petición devuelve **500**. Ése es el problema que el paso 5 resuelve.

Si te atascas

1 · Sigue devolviendo 200 con el nombre.

No paraste contribuyentes del todo, o quedó vivo el proceso. Compruébalo:

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:8201/salud
```

Tiene que dar 000.

2 · Devuelve 503 y habla de tramites.

Ése es el **gateway** diciendo que no pudo hablar con Trámites. Has parado el servicio equivocado.

3.5 Paso 5 · Circuit breaker y degradación

Qué vamos a hacer

Que Trámites siga atendiendo aunque Contribuyentes esté caído, y que **deje de llamarlo**.

Para entenderlo mejor

«*Su expediente está en proceso. No he podido confirmar el nombre del titular en este momento.*»
La oficina atiende. Dice lo que sabe y **marca claramente lo que no pudo confirmar**.

El problema

El 500 del paso 4 es una decisión tomada por omisión: nadie decidió que un fallo de Contribuyentes tumbara la consulta de un trámite. **Simplemente pasó.**

La alternativa, y por qué no

Es el Lab 10 aplicado entre servicios, y con una decisión de negocio encima:

- **Propagar el error** (el 500 del paso 4): honesto en lo técnico, y convierte la caída de un servicio en la de dos.
- **Degradar**, que es lo de aquí: se devuelve el trámite con el nombre marcado como no disponible. **Es una decisión de negocio** — vale porque consultar un trámite no depende de poder mostrar el nombre. Para otro dato podría no valer.
- **Una cola** en vez de una llamada: el mensaje espera a que el otro vuelva. Es la respuesta correcta cuando **nada se puede perder**, y necesita un intermediario de mensajería que la sala del SII no puede tener.

Y encima, el **circuito** del Lab 10: cuando Contribuyentes lleva varios fallos, Trámites **deja de llamarlo**. Ni espera ni lo entorpece mientras se levanta.

Se pega

Lo esencial ya está en el bloque del paso 3 — ahí están los dos `catch`. El de `CallNotPermittedException` es el del circuito abierto, y **no cuesta una llamada de red**: ni se intenta.

Lo que vas a ver

Con Contribuyentes todavía parado:

```
petición 1: NO_DISPONIBLE · HTTP 200 · 0.009944s
petición 2: NO_DISPONIBLE · HTTP 200 · 0.010110s
petición 3: NO_DISPONIBLE · HTTP 200 · 0.006215s
petición 4: NO_DISPONIBLE · HTTP 200 · 0.005993s
petición 5: NO_DISPONIBLE · HTTP 200 · 0.005709s
petición 6: NO_DISPONIBLE · HTTP 200 · 0.005366s
```

De 500 a 200. El trámite se consulta, y el campo que no se pudo confirmar viene marcado como `NO_DISPONIBLE` — no vacío, no inventado: **marcado**.

Y mira los tiempos: **milésimas**. Con el circuito abierto no se está esperando a nadie.

Vuelve a arrancar Contribuyentes y repite: al cabo de unos segundos vuelve el nombre.

Tus tiempos serán otros, y el número de peticiones hasta que el circuito abra también. Lo que tiene que verse es el patrón: **el código pasa a 200 y el tiempo se desploma**.

Vas bien si...

Con contribuyentes parado obtienes **200** con `"estadoDelNombre": "NO_DISPONIBLE"`, y las peticiones tardan milésimas.

Si te atascas

1 · Sigue dando 500.

El `catch` no captura lo que llega. Con el circuito abierto la excepción es `CallNotPermittedException`, que **no** es un fallo de red: si sólo capturas excepciones HTTP, se te escapa.

2 · Da 200 pero tarda segundos.

El circuito no está abriendo: comprueba sus umbrales, como en el Lab 10.

3.6 Paso 6 · El gateway

Qué vamos a hacer

Poner una puerta única delante de los tres servicios.

Para entenderlo mejor

La recepción. El ciudadano no va a cada oficina: va a recepción, **enseña el carnet una vez**, y le dirigen. Las oficinas de dentro no piden carnet a nadie.

El problema

Sin puerta única, el cliente tiene que conocer los tres puertos, y **cada servicio tiene que validar el token por su cuenta**. Eso significa repartir la clave de firma a los tres — y con firma simétrica, cualquiera de ellos podría **fabricar** tokens de administrador.

La alternativa, y por qué no

- **Spring Cloud Gateway**: es lo que se usaría en un proyecto real, y trae reintentos, límites por cliente y circuitos por ruta ya hechos. Aquí lo enseñaría todo menos lo que se quiere ver: sería configuración en un yml y nadie vería ocurrir el reenvío.
- **Una tabla de rutas escrita a mano**, que es lo de aquí: treinta líneas y se lee entera.
- **Descubrimiento de servicios** (Eureka, Consul): necesario en cuanto las instancias dejen de tener puerto fijo, que es lo primero que pasa al desplegar de verdad.

Y una decisión que conviene entender: el gateway REENVÍA, no redirige. Si redirigiera con un 302, el cliente tendría que poder llegar al 8201 y al 8202 — y entonces el gateway deja de ser una puerta y pasa a ser una sugerencia. Con reenvío, los tres servicios pueden estar donde nadie los alcance.

Se pega

La tabla de rutas, en `practica/gateway/src/main/java/cl/dgt/gateway/enrutado/TablaDeRutas.java`:

```
package cl.dgt.gateway.enrutado;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;

@Component
public class TablaDeRutas {

    public record Ruta(String prefijo, String destino, String servicio) {
    }

    private final List<Ruta> rutas;

    public TablaDeRutas(@Value("${lab14.contribuyentes.url}") String contribuyentes,
                        @Value("${lab14.tramites.url}") String tramites,
                        @Value("${lab14.auditoria.url}") String auditoria) {
        this.rutas = List.of(
            new Ruta("/contribuyentes", contribuyentes, "contribuyentes"),
            new Ruta("/tramites", tramites, "tramites"),
            new Ruta("/auditoria", auditoria, "auditoria"));
    }

    public Optional<Ruta> para(String ruta) {
        return rutas.stream().filter(r -> ruta.startsWith(r.prefijo())).findFirst();
    }
}
```

```

public List<Ruta> todas() {
    return rutas;
}
}

```

Tres líneas, y son todo el «enrutado dinámico» que este laboratorio necesita.

Lo que vas a ver

```
curl -s -o /dev/null -w 'sin token -> HTTP%{http_code}\n' localhost:8200/tramites/1
```

```
sin token -> HTTP 401
```

Y con token, la petición llega al servicio de dentro y vuelve. **El ciudadano sólo conoce el 8200.**

Vas bien si...

Sin token, el gateway devuelve 401 **sin llegar a llamar** al servicio de dentro. Con token, la respuesta llega igual que si hubieras llamado al servicio directamente.

Si te atascas

1 · 404 en el gateway para una ruta que existe en el servicio.

El prefijo no está en la tabla de rutas, o no coincide.

2 · El gateway devuelve 503 diciendo que un servicio no contestó.

Ese servicio está parado. Es el gateway haciendo su trabajo: distingue «no existe la ruta» de «la oficina de destino no abre».

3.7 Paso 7 · Seguir una petición por tres servicios

Qué vamos a hacer

Ver el **mismo identificador** en tres consolas distintas.

Para entenderlo mejor

El número de expediente del Lab 11, ahora **cruzando oficinas**. Se apunta en recepción y viaja con la petición a todas las que intervienen.

El problema

Con una sola aplicación, buscar en un log era buscar en un archivo. Con cuatro procesos son cuatro archivos, y **la marca de tiempo no basta** para saber qué línea de un servicio corresponde a qué línea de otro.

La alternativa, y por qué no

- **Buscar por marca de tiempo:** funciona con poco tráfico y deja de funcionar en cuanto hay más de una petición por segundo.
- **El trace id propagado,** que es lo de aquí: cada servicio lee la cabecera, la deja en el MDC y **la reenvía** al siguiente.

- **OpenTelemetry**: hace esto y además da la traza entera con tiempos por tramo. Es lo correcto en un sistema real, y es otra pieza que instalar y operar.

El filtro de correlación está COPIADO en los cuatro servicios, no compartido en una librería. A primera vista parece un descuido y es una decisión: una librería común entre microservicios los vuelve a acoplar — para subir su versión hay que desplegar los cuatro, que es exactamente lo que se estaba intentando evitar. Duplicar treinta líneas sale más barato que ese acoplamiento.

Lo que vas a ver

Haz una petición y mira las tres consolas:

```
gateway      [568f01cb]
tramites     [568f01cb]
contribuyentes [568f01cb]
```

El mismo id, en tres procesos distintos. Con eso, una queja de un ciudadano se sigue de punta a punta.

Vas bien si...

El id que aparece en la consola del gateway aparece igual en la de trámites y en la de contribuyentes.

Si te atascas

1 · Cada servicio muestra un id distinto.

El id no se está **reenviando**: el cliente HTTP de trámites tiene que copiar la cabecera X-Trace-Id en la llamada saliente. Leerla no basta.

2 · Los corchetes salen con puntos.

El filtro no se ejecuta en ese servicio, o falta el patrón en su `application.yml`. Recuerda que **cada servicio tiene el suyo**.

3.8 Paso 8 · Consistencia eventual

Qué vamos a hacer

Emitir un trámite con Auditoría caída, y decidir qué es aceptable.

Para entenderlo mejor

Registrar el expediente cuando la Oficina de Auditoría está cerrada. **¿Se admite el trámite o se rechaza al ciudadano?**

El problema

En una sola aplicación, guardar el trámite y anotar la auditoría iban **en la misma transacción**: o las dos o ninguna. Repartidos, **no hay transacción que abarque a los dos**.

La alternativa, y por qué no

- **Exigir la auditoría:** si Auditoría está caída, no se emiten trámites. Consistente y frágil: una oficina de apoyo tumba la principal.
- **Degradar y perder el registro**, que es lo que hace este lab: el trámite se emite y **la anotación de auditoría se pierde**. Aceptable aquí, y **no lo sería en un sistema tributario de verdad**, donde la traza es obligatoria por ley. Hay que decirlo así de claro.
- **Una cola:** el evento espera a que Auditoría vuelva y no se pierde nada. Es la respuesta correcta, y pide un intermediario de mensajería que necesita Docker.
- **El patrón «outbox»:** guardar el evento **en la misma base y la misma transacción** que el trámite, y que otro proceso lo reenvíe después. Es la variante que **sí** se podría hacer aquí sin Docker, y es el mejor ejercicio para después de este curso.

Vas bien si...

Puedes explicar qué se pierde cuando Auditoría está caída, y nombrar dos formas de no perderlo.

4 Lo que aprendiste

1 · Una base por servicio es lo que hace que sean servicios.

Compartir tablas devuelve el acoplamiento y quita la única ventaja: poder desplegar por separado. Y se paga sin JOIN y sin transacción común.

2 · Cada llamada entre servicios puede fallar, y hay que decidir qué pasa entonces.

De **500 a 200 degradado** con una decisión: el trámite se consulta y el dato que no se pudo confirmar viene **marcado**. Esa decisión no es técnica — la toma quien conoce el trámite.

3 · El gateway es una puerta, no una sugerencia.

Reenvía en vez de redirigir, y valida el token una sola vez. Así los servicios de dentro pueden estar donde nadie los alcance.

4 · Sin un identificador que cruce, los logs de cuatro procesos no se pueden leer juntos.

El mismo [568f01cb] en tres consolas es lo que convierte cuatro archivos sueltos en la historia de una petición.

5 Para profundizar

- **Mata auditoría** en vez de contribuyentes y mira qué cambia. ¿Se emite el trámite?
- **Arranca dos instancias de trámites** en puertos distintos y añádelas a la tabla de rutas. ¿Qué te falta para repartir la carga entre las dos?
- **Quita el reenvío de la cabecera X-Trace-Id** en el cliente de trámites y vuelve a mirar las tres consolas.
- **Diseña el outbox** del paso 8 sobre el papel: qué tabla, qué proceso lo lee, qué pasa si se reenvía dos veces.
- **Cuenta cuántas cosas hay que arrancar** para tener el sistema en pie. Ése es el coste que hay que comparar contra la ventaja.

6 Antes de cerrar

Para los cuatro servicios con Ctrl+C, en sus cuatro terminales. Si alguno quedó vivo:

```
for p in 8200 8201 8202 8203; do lsof -ti:$p | xargs kill -9; done
for p in 55450 55451 55452; do lsof -ti:$p | xargs kill -9; done
```

Lo que te llevas:

Partir un sistema resuelve el despliegue independiente y estrena la red como punto de fallo. Cada llamada entre servicios necesita una decisión sobre qué pasa si falla, y esa decisión es de negocio. Y sin un identificador que cruce, cuatro consolas no se pueden leer juntas.

Y con esto se cierra el arco. Empezaste con una clase que imprimía una frase y terminas con cuatro procesos repartidos que siguen atendiendo cuando uno se cae. Todo lo que hay en medio son las mismas ideas del Lab 02 —declarar lo que necesitas y dejar que otro lo resuelva— aplicadas a problemas cada vez más grandes.